The Subset-sum problem

- Involves searching through a collection of numbers to find a subset of numbers that sums to a particular number

Optimization

It asks, given a collection of non-negative numbers and a non-negative number x, determines the subset, the sum of whose elements will be less than or equal to x.

Subset Sum problem using DP.

Let A be an array or set which contains 'n' non-negative integers. Find a subset 'x' of set 'A' such that sum of all elements of x = w here w is another input (sum)

eg- A = {1, 2, 5, 9, 4} ← non-negative

Sum(w) = 18

Question.

A = {2, 3, 5, 7, 10}

Sum(w) = 14

i ↓ \ Sum(j) →	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0 (2)	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1 (3)	1	0	1	1	0	1	0	0	0	0	0	0	0	0	0
2 (5)	1	0	1	1	0	1	0	1	1	0	1	0	0	0	0
3 (7)	1	0	1	1	0	1	0	1	1	1	1	0	1	0	1
4 (10)	1	0	1	1	0	1	0	1	1	1	1	0	1	1	1

m[i][j] = 1 ← when m[i][j] will be one

cases

① A[i] = j

② A[i-1][j] = 1

③ A[i-1][j - A[i]] = 1

Question. Which of the following is/are properties of a dynamic programming problem?

Ans optimal substructure and overlapping subproblems.

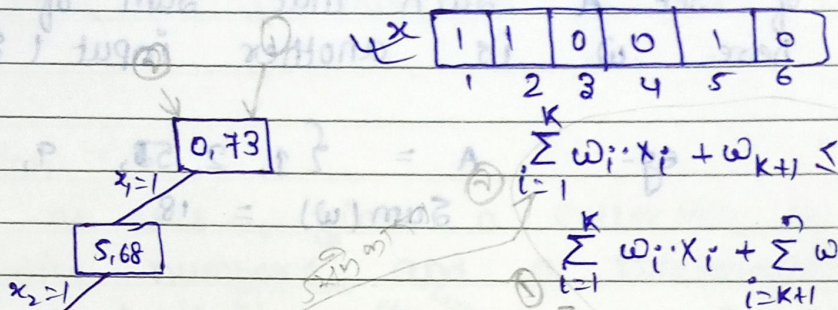
⇒ if an optimal soln can be created for a problem by constructing optimal soln for its subproblems, the problem possess optimal substructure property.

Sum of Subsets Problem (Backtracking)

$w[1:6] = \{5, 10, 12, 13, 15, 18\}$

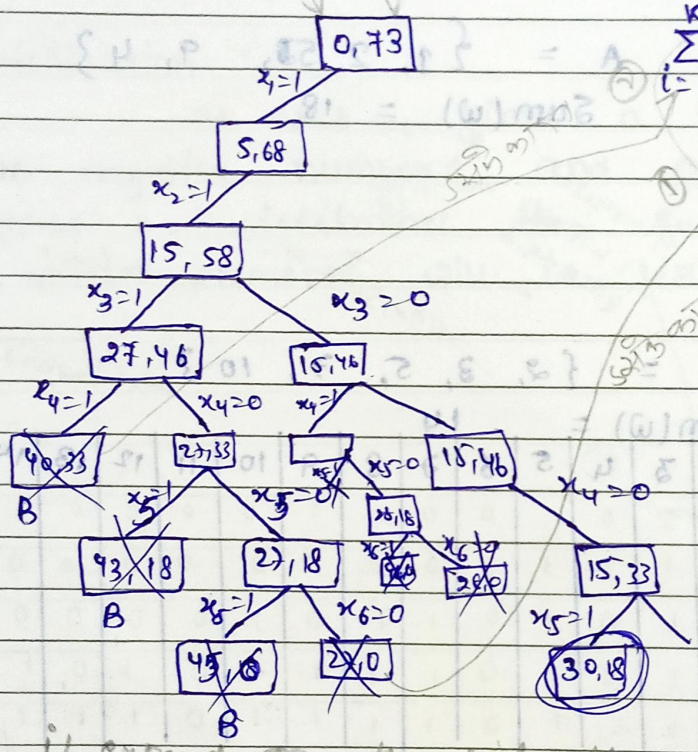
$n = 6 \quad m = 30$

Solution



$$\sum_{i=1}^{k} w_i \cdot x_i + w_{k+1} \leq m$$

$$\sum_{i=1}^{k} w_i \cdot x_i + \sum_{i=k+1}^{n} w_i > m$$



if expand on other side then more suitable answer we get

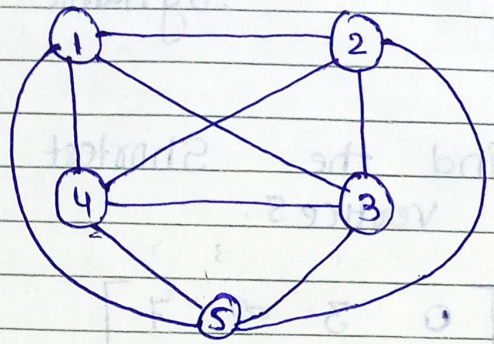
Floyd-Warshall algorithm is also called as Floyd's algorithm, Roy-Floyd algorithm, Roy-Warshall algorithm, or WFI algorithm ⇒ follows dynamic approach.

⇒ Dijkstra and Bellman-ford are both single-source, shortest-path algorithms. This means that they only compute the shortest path from a single source.

we avoid backtracking for time consuming in which we have to identified the cost of every path

Traveling Salesman problem (Branch and Bound).

	1	2	3	4	5
1	∞	20	30	10	11
2	15	∞	16	4	2
3	3	5	∞	2	4
4	19	6	18	∞	3
5	16	4	7	16	∞



Solution

Reduced matrix

	∞	10	17	0	1
	12	∞	11	2	0
	0	3	∞	0	2
	15	3	12	∞	0
	11	0	0	12	∞
	1	0	3	0	0

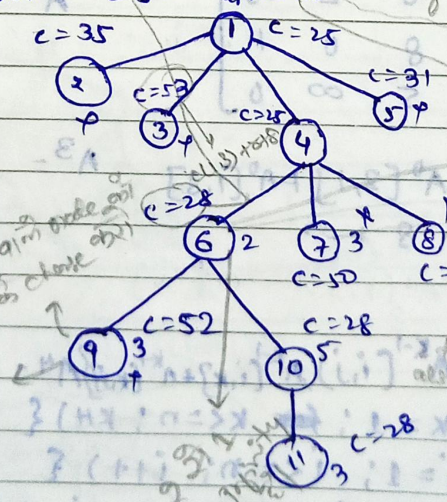
10, 2, 2, 3, 4

21 + 4 = 25

reduced cost = 25.

	∞	∞	∞	∞	∞
	∞	∞	11	2	0
	0	∞	∞	0	2
	15	∞	12	∞	0
	11	∞	0	12	∞

c(1,2) + r + r'
10 + 25 + 0



	∞	∞	∞	∞	∞
	∞	∞	11	∞	0
	0	∞	∞	∞	2
	∞	∞	∞	∞	∞
	11	∞	0	∞	∞

6

	∞	∞	∞	∞	∞
	∞	∞	∞	∞	∞
	∞	∞	∞	∞	0
	∞	∞	∞	∞	∞
	0	∞	∞	∞	∞

2, 11

1 - 4 - 2 - 5 - 3 - 1

shortest distance

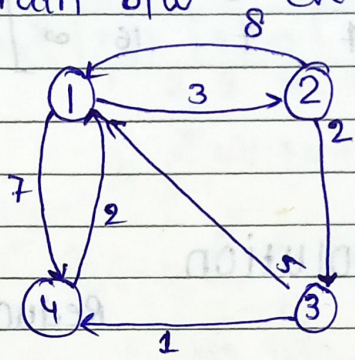
DP says that a problem should be solved by a ~~series~~ of decision sequence

does not work for graphs with negative cycles

All pairs Shortest Path (Floyd-Warshall) Dynamic programming.

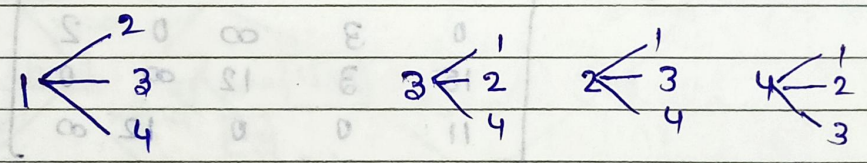
work for both directed and undirected weighted graph

Q. Find the Shortest Path b/w every pair of vertices.



$$A^0 = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 8 \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix}$$

Solution



this is same as A^0

$$A^1 = \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{bmatrix}$$

this is same as A^1

$$A^2 = \begin{bmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

$A^0[2,3] = A^0[2,1] + A^0[1,3]$

$2 < 8 + \infty$

$$A^3 = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 12 & 5 & 7 & 0 \end{bmatrix}$$

formula

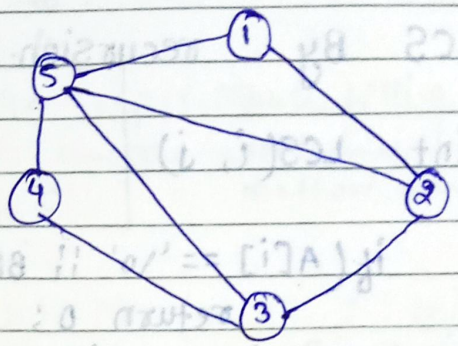
$A^k[i,j] = \min\{A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j]\}$

$$A^4 = \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix}$$

```
for (k = 1; k <= n; k++) {
    For (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            A[i,j] = min(A[i,j], A[i,k] + A[k,j]);
        }
    }
}
```

time complexity $O(n^3)$

Graph Coloring (Backtracking)



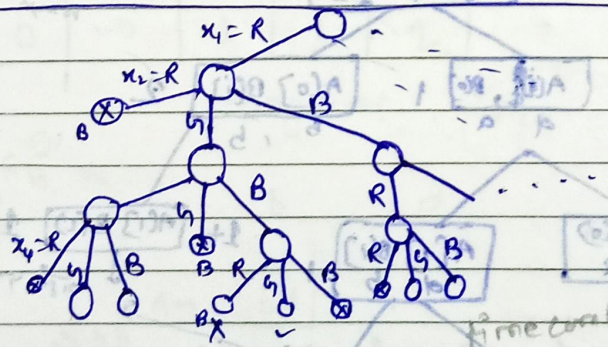
$m = 3$
R, G, B

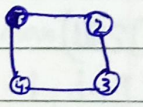
Solution

1	2	3	4	5
R	G	R	G	B
G	R	G	R	B
⋮	⋮	⋮	⋮	⋮

etc

m Coloring decision problem
m Coloring optimization problem → minimum colors are required for coloring

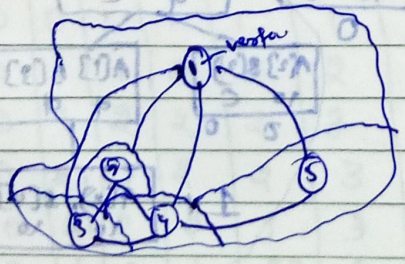


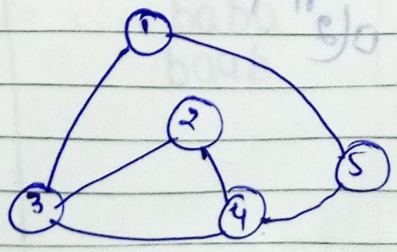


$m = 3$
{R, G, B}

1) R, G, R G
2) R, G, R, B
3) R, G, B, G
4) R, B, R B
5) R, B, R, G

Application





Recursion and Dynamic programming can be used to solve the longest common subsequence sub problem

Longest Common Subsequence (LCS)

LCS By recursion

```
int LCS(i, j)
{
    if(A[i] == '\0' || B[j] == '\0')
        return 0;
    else if (A[i] == B[j])
        return 1 + LCS(i+1, j+1);
    else
        return max(LCS(i+1,j), LCS(i, j+1));
}
```

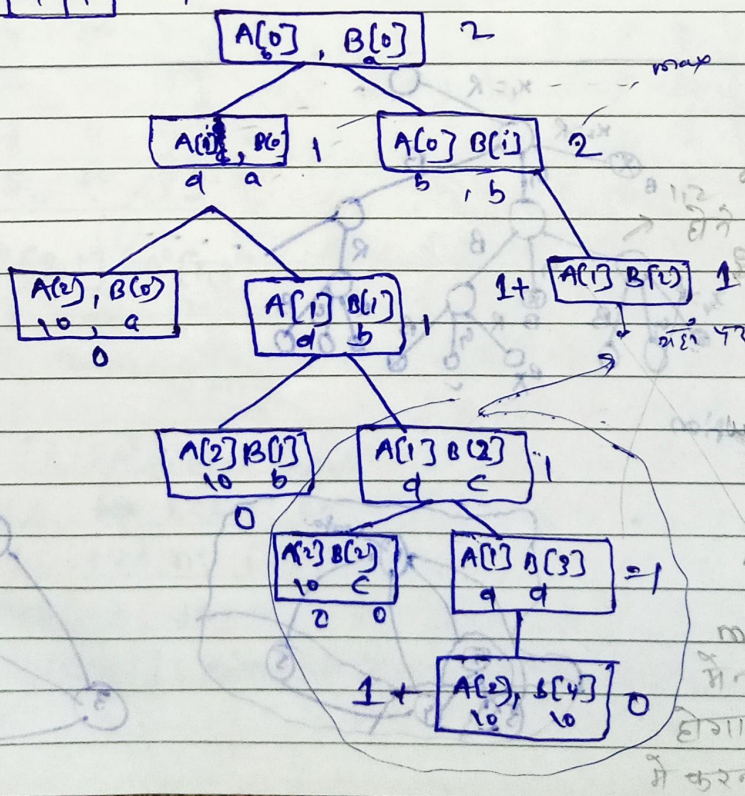
recursion are time consuming.

3.

A | b | a | \0 | (0, 1, 2) — m

B | a | b | c | a | \0 | n

this is top-down approach



time complexity

$O(2^n)$

* Character from a sequence that appear in the same relative order but not necessarily contiguous. (2^m no. of sub-sequences for a sequence of length m).

by memoization.

↳ (matching should not intersect each other)

		a	b	c	d	10
		0	1	2	3	4
b	1	2	2			
d	2	1	1	1	1	
10	3	0	0	0		0

Table filling bottom to top but plotted method was top down.

Note: Memoization reduce the function call.

memoization taking a time of (m x n) where m and n are no. of element

LCS By Dynamic programming

A: 5 4 (1 2)

B: a b c 4 (1 2 3 4)

* DP follows Bottom up approach.

table was filling top to down

		a	b	c	d
	0	1	2	3	4
0	0	0	0	0	0
b 1	0	0	1	1	1
d 2	0	0	1	1	2

1 diagonal element

initially filled with 0

diagonal

size = 2

b d

```
if(A[i] == B[j])
    LCS[i,j] = 1 + LCS[i-1, j-1]
else
    LCS[i,j] = max(LCS[i-1,j], LCS[i, j-1])
```

time complexity O(m x n)

Ans | bd

X = abaaba Y = babbab

(4)

baba
baab

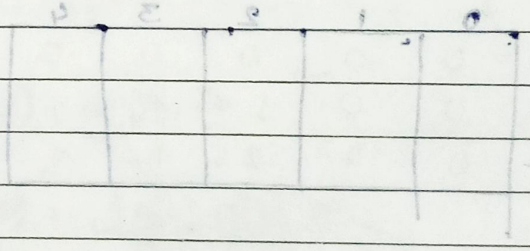
(i) ↓	y→(j)	^	b	a	b	b	a	b
	^	0	0	0	0	0	0	0
a		0	0	1	1	1	1	1
b		0	1	1	2	2	2	2
a		0	1	2	2	2	3	3
a		0	1	2	2	2	3	3
b		0	1	2	3	3	3	4
a		0	1	2	1	3	4	4

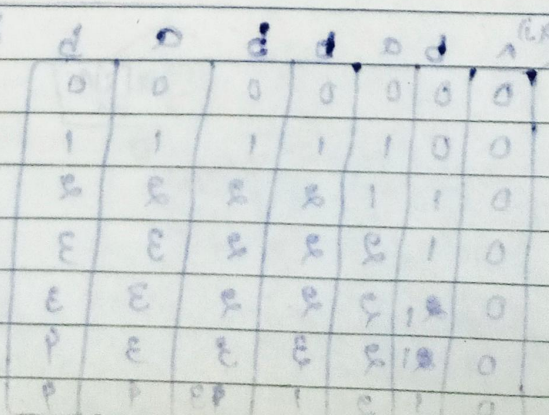
* The naive algorithm will take $O(n \cdot 2^m)$ time.
  - There are 2^m subsequences of X.
  - Testing sequences whether or not is a subsequence of Y takes $O(n)$ time

$X = x_1 \ldots x_m$
$Y = y_1 \ldots y_n$

* There are multiple approach to implement dp. ie bottom-up approach, top-down approach

* `char[] x = s1.toCharArray();`





0/1 Knapsack problem

$m = 8$ $\quad P = \{1, 2, 5, 6\}$

$n = 4$ $\quad \omega = \{2, 3, 4, 5\}$

by tabulation method.

P	ω	0	0	1	2	3	4	5	6	7	8
			0	0	0	0	0	0	0	0	0 → Profit
1	2	1	0	0	1	1	1	1	1	1	1
2	3	2	0	0	1	2	2	3	3	3	3
5	4	3	0	0	1	2	5	6	6	7	7
6	5	4	0	0	1	2	5	6	6	7	8

↕ weight

Formula.

$$v[i, \omega] = \max\{v[i-1, \omega],\ v[i-1, \omega - \omega[i]] + P[i]\}$$

$$v[4,1] = \max\{v[3,1],\ v[3,1-5] + 6\}$$

$$v[4,5] = \max\{v[3,5],\ v[3,5-5] + 6\}$$
$$5, \quad 0+6$$

$$v[4,6] = \max\{[v[3,6],\ v[3,6-5] + 6]$$
$$6, \quad 0+6$$

$$v[4,7] = \max\{v[3,7],\ v[3,7-5] + 6\}$$
$$7, \quad 1+6$$

$$v[4,8] = \max\{v[3,8],\ v[3,8-5] + 6$$
$$7, \quad 2+6$$

Ans. $x_1 \ x_2 \ x_3 \ x_4$ $\quad 8-6=2$

$0 \ \ 1 \ \ 0 \ \ 1$

sets method

dominance rule

II method

$S^0 = \{(0,0)\}$ $\quad S_1^0 = \{(1,2)\}$

$S^1 = \{(0,0), (1,2)\}$ $\quad S_1^1 = \{(2,3), (3,5)\}$

$S^2 = \{(0,0)\ (1,2)\ (2,3)\ (3,5)\}$

$S_1^2 = \{(5,4), (6,6), (7,7), (8,8)\}$

$S^3 = \{(0,0)\ (1,2)\ (2,3)\ (3,5)\ (5,4)\ (6,6)\ (7,7)\}$

$S_1^3 = \{(6,5)\ (7,7)\ (8,8)\ (11,9)\ (12,11)\ (13,12)\}$

$S^4 = \{(0,0)\ (1,2)\ (2,3)\ (5,4)\ (6,6)\ (6,5)\ (7,7)\ (8,8)\}$

$(8,8) \in S^4$

but $(8,8) \notin S^3 \quad \therefore x_4 = 1$

$(8-6, 8-5) = 2,3$

(2) $(2,3) \in S^3$

but $(2,3) \in S^2 \quad x_3 = 0$

(3) $(2,3) \in S^2$

but $(2,3) \notin S^1 \quad x_2 = 1$

$(2-2, 3-3) = (0,0)$

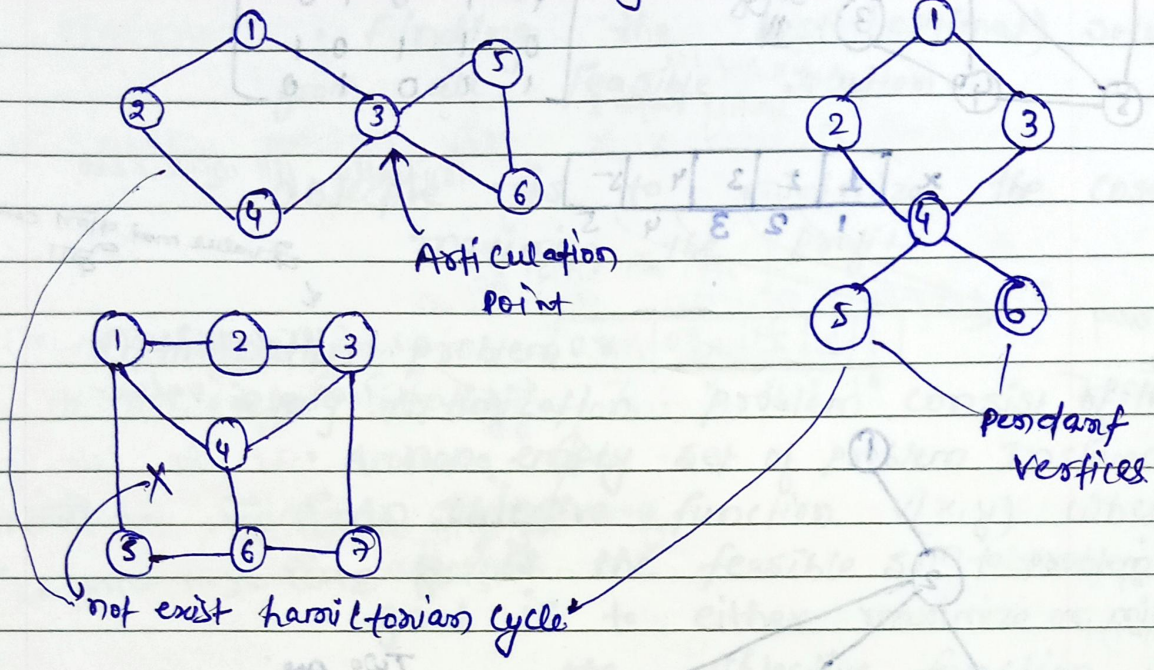
(4) $(0,0) \in S^1$ and $(0,0) \in S^0 \quad \therefore x_1 = 0$

x_1	x_2	x_3	x_4
0	1	0	1

* if you change the starting point then that is not another graph

NP hard problem

Hamiltonian Cycles.



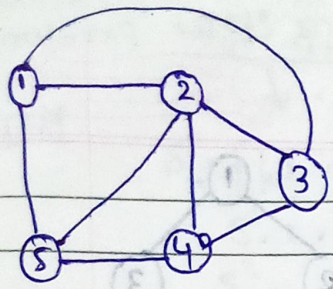
Solution by using Backtracking.

```
Algorithm Hamiltonian(k) {
    do {
        Nextvertex(k);
        if(x[k] == 0)    return;
        if( k == n)      print(x[1:n]);
        else             Hamiltonian(k+1);
    } while(true);
}

Algorithm Nextvertex(k) {
    do {
        x[k] = (x[k] + 1) mod (n+1);
        if( x[k] == 0 )  return;
        if( G[x[k-1], x[k]] ≠ 0) {
            for j = 1 to k-1 do if (x[j] == x[k]) break;
            if( j == k )
                if( k < n or (k == n) and G[x[n], x[1]] ≠ 0
    } while(true);
}
```

if there any edge from previous vertex checking for Duplicate

if there is a last vertex then there is an edge to the one



avoid of duplicate cycle.

find out the starting vertex 1

$$G = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 \end{bmatrix}$$

x

1	2	3	4	5
1	2	3	4	5

initially fill with zero

value mod वाला code दूँगा.

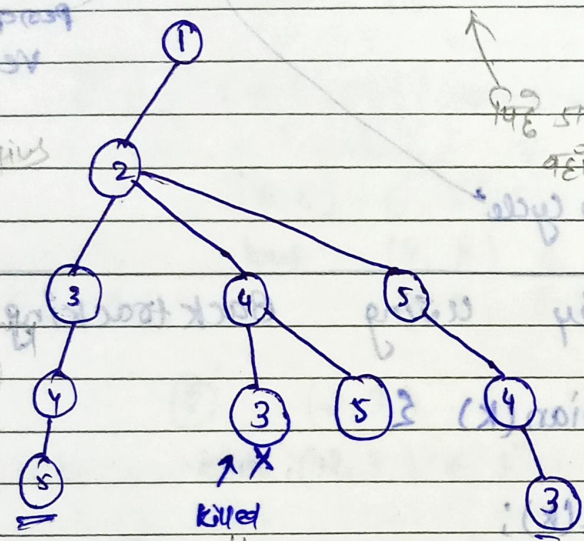
1	2	3	4	5
0	1	2	3	4	5	0	0	1	2	3	4	5	0

for every value 0 1 2 3 4 5

4!

for n vertices (n-1)!

time complexity $O(n^n)$ n! =



Two Ans

① 1, 2, 3, 4, 5, 1

② 1, 2, 5, 4, 3, 1

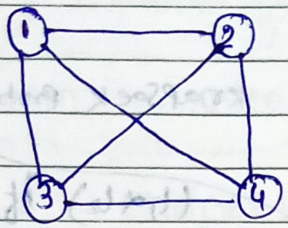
and expand more further.

things checked in bounding function

1. should not take duplicate values
2. whenever you take any vertex there should be an edge from the previous vertex.
3. last vertex should be an edge from the first vertex.

Optimization < maximum / minimum

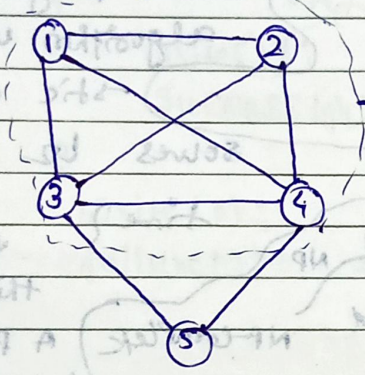
Clique Decision Problem (CDP).



complete graph.

$|V| = n$

$|E| = \frac{n(n-1)}{2}$.

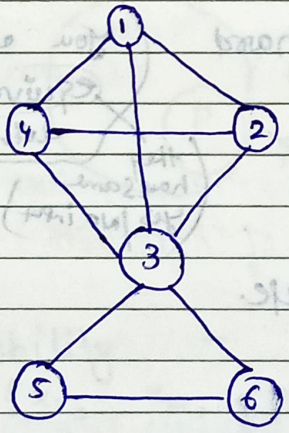


→ clique.

Sub graph of a graph.

Decision Problem

finding out whether the answer is true or false.



$K = 4$

$K = 3$

$K = 2$

Decision problem :- find if a graph is having a clique of size k.

Optimization problem :- find max clique in a graph.

Sat ∝ CDP

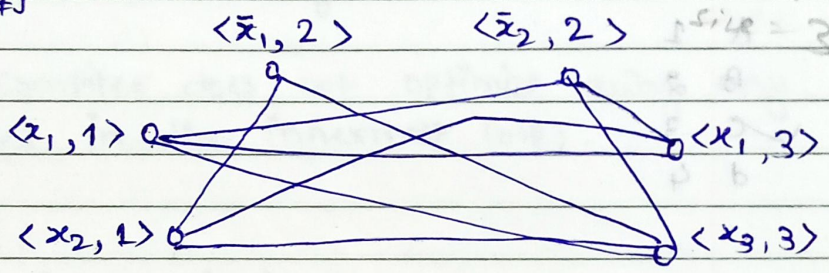
x_1, x_2, x_3

$F = \bigwedge_{i=1}^{k} C_i$ $\quad F = (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2) \wedge (x_1 \vee x_3)$ — 1

$C_1 = 1$ $\quad C_2 = 1$

$G = \{(\langle a,i\rangle, \langle b,j\rangle) \mid i \neq j$ and $b \neq \bar{a}\}$

max clique size = 3



$K = 3$

Initially all $x_1 x_2 x_3$ are false

x_1	x_2	x_3
0	1	1

Prove that CDP is NP-hard

Characteristic of Hash functions

① Map larger values to smaller values.
② Should be O(1) or O(len) for String keys
③ Should be uniformly distribute large keys into hash table slots.

Eg. of Hash functions

1. for large keys

$$h(key) = key \,\%\, m$$

2. for String keys :- Weight Sum

"cat" | "act"

$$(s[0] \times x^0 + x^1 * s[1] + x^2 \times s[2]) \,\%\, m$$

↳ any prime no.

3. for objects

weighted sum of elements

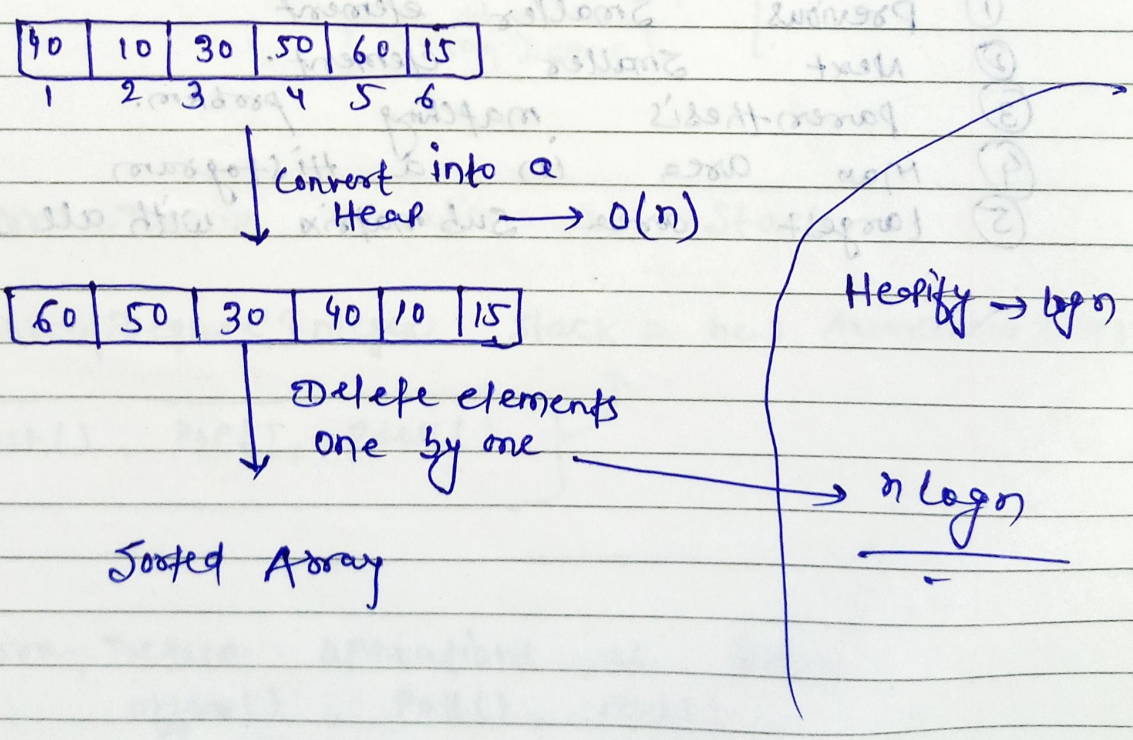
एक array को traverse करते जाओ और maxHeap बनाते जाओ उस वेस में time complexity O(nlogn)

⇓

इसे वे से नही create करना है इसे क्योंकि इससे कि अच्छा T.C O(n) वाला use करना होता है।

↳ Heapify

① एक maxHeap / minHeap कि ~~कई~~ एक से अधिक config -uration हो सकते है यदि array मे element का index अलग - अलग हो सकता है।



Stack एक concrete class है।

```
Deque<Integer> s = new ArrayDeque<>();
      ,,       ,,       ,,  LinkedList<>();
```

Benefit:1 Deque का use करके stack को implement करो क्योंकि deque एक interface है जो कि array भी implement करता है और linked list भी करता है।

2. Stack internally vector use कर रहा होता है जो कि element को fetch करने या delete करने के लिए index का use कर रहा होता है।

Question.

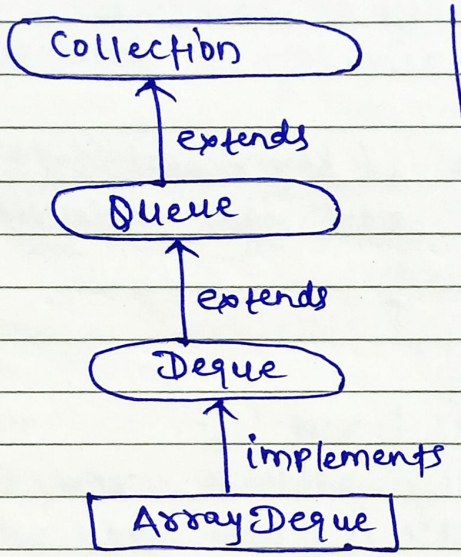
1. Previous Smaller element
2. Next Smaller element.
3. Parenthesis matching problem.
4. Max area in a Histogram
5. Largest area Sub matrix with all 1's

Deque (Doubly Ended queue)

getFirst() , getLast() , removeFirst() removeLast()
addFirst() , addLast()

Note:- ArrayDeque operations as Doubly ended queue.

throw Exception



Not throw exception

offerFirst()
offerLast()
peekFirst()
peekLast()
pollFirst()
pollLast()

ArrayDeque Operation as Stack

```
ArrayDeque<Integer> stack = new ArrayDeque<>();
```

push() , pop() , peek()

ArrayDeque operations as Queue

→ offer() , poll() , peek()

```
ArrayDeque<Integer> queue = new ArrayDeque<>();
```